

# Generalized Optimization Modulo Theories

Nestan Tsiskaridze[1(✉)], Clark Barrett[1], and Cesare Tinelli[2]

[1] Stanford University, Stanford, CA, USA
{nestan,barrett}@cs.stanford.edu
[2] The University of Iowa, Iowa City, IA, USA
cesare-tinelli@uiowa.edu

**Abstract.** Optimization Modulo Theories (OMT) has emerged as an important extension of the highly successful Satisfiability Modulo Theories (SMT) paradigm. The OMT problem requires solving an SMT problem with the restriction that the solution must be optimal with respect to a given objective function. We introduce a generalization of the OMT problem where, in particular, objective functions can range over partially ordered sets. We provide a formalization of and an abstract calculus for the Generalized OMT problem and prove their key correctness properties. Generalized OMT extends previous work on OMT in several ways. First, in contrast to many current OMT solvers, our calculus is theory-agnostic, enabling the optimization of queries over any theories or combinations thereof. Second, our formalization unifies both single- and multi-objective optimization problems, allowing us to study them both in a single framework and facilitating the use of objective functions that are not supported by existing OMT approaches. Finally, our calculus is sufficiently general to fully capture a wide variety of current OMT approaches (each of which can be realized as a specific strategy for rule application in the calculus) and to support the exploration of new search strategies. Much like the original abstract DPLL(T) calculus for SMT, our Generalized OMT calculus is designed to establish a theoretical foundation for understanding and research and to serve as a framework for studying variations of and extensions to existing OMT methodologies.

**Keywords:** Optimization Modulo Theories (OMT) · Optimization · Satisfiability Modulo Theories (SMT) · Abstract Calculus

## 1 Introduction

Over the past decade, the field of Optimization Modulo Theories (OMT) has emerged, inspiring the interest of researchers and practitioners alike. OMT builds on the highly successful Satisfiability Modulo Theories (SMT) [3] paradigm and extends it: while the latter focuses solely on finding a theory model for a first-order formula, the former adds an objective term that must be optimized with respect to some total ordering over the term's domain.

The development of OMT solvers has fostered research across an expanding spectrum of applications, including scheduling and planning with resources [7,

© The Author(s) 2024
C. Benzmüller et al. (Eds.): IJCAR 2024, LNAI 14739, pp. 458–479, 2024.
https://doi.org/10.1007/978-3-031-63498-7_27

13,17,20,26,30,35,38,48,58], formal verification and model checking [37,49], program analysis [10,23,25,28,69], requirements engineering and specification synthesis [21,41–43], security analysis [4,18,46,61], system design and configuration [14,15,29,34,47,51,63,68], machine learning [59,62], and quantum annealing [5].

Various OMT procedures have been developed for different types of optimization objectives (e.g., single- and multi-objective problems), underlying theories (e.g., arithmetic and bitvectors), and search strategies (e.g., linear and binary search). We provide an overview of established OMT techniques in Sect. 5. An extensive survey can be found in Trentin [64].

We introduce a proper generalization of the OMT problem and an abstract calculus for this generalization whose main goal is similar to that of the DPLL($T$) calculus for SMT [45]: to provide both a foundation for theoretical understanding and research and a blueprint for practical implementations. Our approach is general in several ways. First, in contrast to previous work in OMT, it is parameterized by the optimization order, which does not need to be total, and it is not specific to any theory or optimization technique, making the calculus easily applicable to new theories or objective functions. Second, it encompasses both single- and multi-objective optimization problems, allowing us to study them in a single, unified framework and enabling combinations of objectives not covered in previous work. Third, it captures a wide variety of current OMT approaches, which can be realized as instances of the calculus together with specific strategies for rule application. Finally, it provides a framework for the exploration of new optimization strategies.

**Contributions .** To summarize, our contributions include:

- a formalization of a generalization of OMT to partial orders that unifies traditional single- and multi-objective optimization problems;
- a theory-agnostic abstract calculus for Generalized OMT that can also be used to describe and study previous OMT approaches;
- a framework for understanding and exploring search strategies for Generalized OMT; and
- proofs of correctness for important properties of the calculus.

The rest of the paper is organized as follows. Section 2 introduces background and notation. Section 3 defines the Generalized OMT problem. Section 4 presents the calculus, provides an illustrative example of its use and addresses its correctness[1]. Finally, Sect. 5 discusses related work, and Sect. 6 concludes.

## 2 Background

We assume the standard many-sorted first-order logic setting for SMT, with the usual notions of signature, term, formula, and interpretation. We write $\mathcal{I} \models \phi$

[1] Full proofs and an additional example are provided in the longer version of this paper [67].

**Table 1.** Theory-specific notation.

| Syntax | Semantics | Meaning |
|---|---|---|
| $\mathsf{Bool}, \mathsf{Int}, \mathsf{Real}, BV_{[n]}, \mathsf{Str}$ | | Sorts for Booleans, integers, reals, bitvectors of length $n$, and character strings |
| $+, -, \times, \div$ | | Arithmetic operators over reals/integers |
| $<_{\mathsf{R}}, >_{\mathsf{R}}, \leq_{\mathsf{R}}, \geq_{\mathsf{R}}$ | $\prec_{\mathsf{R}}, \succ_{\mathsf{R}}, \preccurlyeq_{\mathsf{R}}, \succcurlyeq_{\mathsf{R}}$ | Comparison operators over reals |
| $<_{\mathsf{Int}}, >_{\mathsf{Int}}, \leq_{\mathsf{Int}}, \geq_{\mathsf{Int}}$ | $\prec_{\mathsf{Int}}, \succ_{\mathsf{Int}}, \preccurlyeq_{\mathsf{Int}}, \succcurlyeq_{\mathsf{Int}}$ | Comparison operators over integers |
| $+_{[n]}, -_{[n]}, \times_{[n]}, \div_{[n]}$ | | Arithmetic modulo $2^n$ operators |
| $<_{[n]}, >_{[n]}, \leq_{[n]}, \geq_{[n]}$ | $\prec_{[n]}, \succ_{[n]}, \preccurlyeq_{[n]}, \succcurlyeq_{[n]}$ | (Unsigned) comparison operators over $BV_{[n]}$ terms |
| $ite(c, x, y)$ | | If-then-else operator (if $c$ then $x$ else $y$) |
| $tup(t_1, \ldots, t_n)$ | | $n$-ary tuple where element $i$ is $t_i$ |
| $<_{\mathsf{str}}, >_{\mathsf{str}}, \leq_{\mathsf{str}}, \geq_{\mathsf{str}},$ | $\prec_{\mathsf{str}}, \succ_{\mathsf{str}}, \preccurlyeq_{\mathsf{str}}, \succcurlyeq_{\mathsf{str}}$ | Strict and non-strict lexicographic and reverse lexicographic orders on strings |
| $\epsilon$ | | The empty string |
| $x \cdot y$ | | String concatenation operator |
| $\mathsf{len}(x)$ | | String length operator |
| $\mathsf{contains}(x, y)$ | | String containment operator (true iff $y$ is a substring of $x$) |

to mean that formula $\phi$ holds in or is *satisfied* by an interpretation $\mathcal{I}$. A *theory* is a pair $\mathcal{T} = (\Sigma, \mathbf{I})$, where $\Sigma$ is a signature and $\mathbf{I}$ is a class of $\Sigma$-interpretations. We call the elements of $\mathbf{I}$ *$\mathcal{T}$-interpretations*. We write $\Gamma \models_{\mathcal{T}} \phi$, where $\Gamma$ is a formula (or a set of formulas), to mean that $\Gamma$ *$\mathcal{T}$-entails* $\phi$, i.e., every $\mathcal{T}$-interpretation that satisfies (each formula in) $\Gamma$ satisfies $\phi$ as well. For convenience, for the rest of the paper, *we fix a background theory $\mathcal{T}$ with equality and with signature $\Sigma$.* We also fix an infinite set $\mathcal{X}$ of sorted variables with sorts from $\Sigma$ and assume $\prec_{\mathcal{X}}$ is some total order on $\mathcal{X}$. We assume that all terms and formulas are $\Sigma$-terms and $\Sigma$-formulas with free variables from $\mathcal{X}$. Since the theory $\mathcal{T}$ is fixed, we will often abbreviate $\models_{\mathcal{T}}$ as $\models$ and consider only interpretations that are $\mathcal{T}$-interpretations assigning a value to every variable in $\mathcal{X}$. At various places in the paper, we use sorts and operators from standard SMT-LIB theories such as integers, bitvectors, strings,[2] or data types [2]. We assume that every $\mathcal{T}$-interpretation interprets them in the same (standard) way. Table 1 lists theory symbols used in this paper and their meanings. A $\Sigma$-formula $\phi$ is *satisfiable* (resp., *unsatisfiable*) *in* $\mathcal{T}$ if it is satisfied by some (resp., no) $\mathcal{T}$-interpretation.

Let $s$ be a $\Sigma$-term. We denote by $s^{\mathcal{I}}$ the value of $s$ in an interpretation $\mathcal{I}$, defined as usual by recursively determining the values of sub-terms. We denote by $FV(s)$ the set of all variables occurring in $s$. Similarly, we write $FV(\phi)$ to denote the set of all the free variables occurring in a formula $\phi$. If $FV(\phi) = \{v_1, \ldots, v_n\}$, where for each $i \in [1, n), v_i \prec_{\mathcal{X}} v_{i+1}$, then the relation *defined by* $\phi$ (in $\mathcal{T}$) is $\{(v_1^{\mathcal{I}}, \ldots, v_n^{\mathcal{I}}) \mid \mathcal{I} \models \phi$ for some $\mathcal{T}$-interpretation $\mathcal{I}\}$. A relation is *definable in* $\mathcal{T}$ if there is some formula that defines it. Let $\boldsymbol{v}$ be a tuple of variables $(v_1, \ldots, v_n)$,

[2] For simplicity, we assume strings are over characters ranging only from `'a'` to `'z'`.

and let $\boldsymbol{t} = (t_1, \ldots, t_n)$ be a tuple of $\Sigma$-terms, such that $t_i$ and $v_i$ are of the same sort for $i \in [1, n]$; then, we denote by $s[\boldsymbol{v} \leftarrow \boldsymbol{t}]$ the term obtained from $s$ by simultaneously replacing each occurrence of variable $v_i$ in $s$ with the term $t_i$.

If $S$ is a finite *sequence* $(s_1, \ldots, s_n)$, we write $\textsc{Top}(S)$ to denote, $s_1$, the first element of $S$ in $S$; we write $\textsc{Pop}(S)$ to denote the subsequence $(s_2, \ldots, s_n)$ of $S$. We use $\emptyset$ to denote both the empty set and the empty sequence. We write $s \in S$ to mean that $s$ occurs in the sequence $S$, and write $S \circ S'$ for the sequence obtained by appending $S'$ at the end of $S$.

We adopt the standard notion of *strict partial order* $\prec$ on a set $A$, that is, a relation in $A \times A$ that is irreflexive, asymmetric, and transitive. The relation $\prec$ is a *strict total order* if, in addition, $a_1 \prec a_2$ or $a_2 \prec a_1$ for every pair $a_1, a_2$ of distinct elements of $A$. As usual, we will call $\prec$ *well-founded* over a subset $A'$ of $A$ if $A'$ contains no infinite descending chains. An element $m \in A$ is *minimal (with respect to $\prec$)* if there is no $a \in A$ such that $a \prec m$. If $A$ has a unique minimal element, it is called a *minimum*.

## 3 Generalized Optimization Modulo Theories

We introduce a formalization of the *Generalized Optimization Modulo Theories* problem which unifies single- and multi-objective optimization problems and lays the groundwork for the calculus presented in Sect. 4.

### 3.1 Formalization

For the rest of the paper, we fix a theory $\mathcal{T}$ with some signature $\Sigma$.

**Definition 1 (Generalized Optimization Modulo Theories (GOMT)).** *A Generalized Optimization Modulo Theories problem is a tuple $\mathcal{GO} := \langle t, \prec, \phi \rangle$, where:*

- *$t$, a $\Sigma$-term of some sort $\sigma$, is an objective term to optimize;*
- *$\prec$ is a strict partial order definable in $\mathcal{T}$, whose defining formula has two free variables, each of sort $\sigma$; and*
- *$\phi$ is a $\Sigma$-formula.*

For any GOMT problem $\mathcal{GO}$ and $\mathcal{T}$-interpretations $\mathcal{I}$ and $\mathcal{I}'$, we say that:

- $\mathcal{I}$ is *$\mathcal{GO}$-consistent* if $\mathcal{I} \models \phi$;
- $\mathcal{I}$ *$\mathcal{GO}$-dominates* $\mathcal{I}'$, denoted by $\mathcal{I} <_{\mathcal{GO}} \mathcal{I}'$, if $\mathcal{I}$ and $\mathcal{I}'$ are $\mathcal{GO}$-consistent and $t^{\mathcal{I}} \prec t^{\mathcal{I}'}$; and
- $\mathcal{I}$ is a *$\mathcal{GO}$-solution* if $\mathcal{I}$ is $\mathcal{GO}$-consistent and no $\mathcal{T}$-interpretation $\mathcal{GO}$-dominates $\mathcal{I}$.

Informally, the term $t$ represents the *objective function*, whose value we want to optimize. The order $\prec$ is used to compare values of $t$, with a value $a$ being considered *better* than a value $a'$ if $a \prec a'$. Finally, the formula $\phi$ imposes constraints on the values that $t$ can take. It is easy to see that the value of

$t^{\mathcal{I}}$ assigned by a $\mathcal{GO}$-solution $\mathcal{I}$ is always minimal. As a special case, if $\prec$ is a total order, then $t^{\mathcal{I}}$ is also unique (i.e., it is a minimum). Once we have fixed a GOMT problem $\mathcal{GO}$, we will informally refer to a $\mathcal{GO}$-consistent interpretation as a *solution (of $\phi$)* and to a $\mathcal{GO}$-solution as an *optimal solution*.

Our notion of Generalized OMT is closely related to one by Bigarella et al. [6], which defines a notion of OMT for a generic background theory using a predicate that corresponds to a total order in that theory. Definition 1 generalizes this in two ways. First, we allow partial orders, with total orders being a special case. One useful application of this generalization is the ability to model multi-objective problems as single-objective problems over a suitable partial order, as we explain below. Second, we do not restrict $\prec$ to correspond to a predicate symbol in the theory. Instead, any partial order *definable* in the theory can be used. This general framework captures a large class of optimization problems.

*Example 1.* Suppose $\mathcal{T}$ is the theory of real arithmetic with the usual signature. Let $\mathcal{GO} := \langle x + y, \prec, 0 < x \wedge xy = 1 \rangle$, where $x$ and $y$ are variables of sort $\mathsf{Real}$ and $\prec$ is defined by the formula $v_1 <_{\mathsf{R}} v_2$ (where $v_1 \prec_{\mathcal{X}} v_2$). A $\mathcal{GO}$-solution is any interpretation that interprets $x$ and $y$ as 1.

*Example 2.* With $\mathcal{T}$ now being the theory of integer arithmetic, let $\mathcal{GO} = \langle x, \prec, x^2 < 20 \rangle$, where $x$ is of sort $\mathsf{Int}$, and $\prec$ is defined by $v_1 >_{\mathsf{Int}} v_2$ (where $v_1 \prec_{\mathcal{X}} v_2$). A $\mathcal{GO}$-solution must interpret $x$ as the maximum integer satisfying $x^2 < 20$ (i.e., $x$ must have value 4).

The examples above are both instances of what previous work refers to as single-objective optimization problems [64], with the first example being a minimization and the second a maximization problem. The next example illustrates a less conventional ordering.

Note that from now on, to keep the exposition simple, we define partial orders $\prec$ appearing in $\mathcal{GO}$ problems only *semantically*, i.e., formally, but without giving a specific defining formula. However, it is easy to check that all orders used in this paper are, in fact, definable in a suitable $\mathcal{T}$.

*Example 3.* Let $\mathcal{GO} = \langle x, \prec, x^2 < 20 \rangle$ be a variation of Example 2, where now, for any integers $a$ and $b$, $a \prec b$ iff $|b| \prec_{\mathsf{Int}} |a|$. A $\mathcal{GO}$-solution can interpret $x$ either as 4 or $-4$. Neither solution dominates the other since their absolute values are equal.

We next show how multi-objective problems are also instances of Definition 1.

### 3.2 Multi-objective Optimization

We use the term *multi-objective optimization* to refer to an optimization problem consisting of several sub-problems, each of which is also an optimization problem. A multi-objective optimization may also require specific interrelations among its sub-problems. In this section, we define several varieties of multi-objective optimization problems and show how each can be realized using Definition 1.

For each, we also state a correctness proposition which follows straightforwardly from the definitions.

In the following, given a strict ordering $\prec$, we will denote its reflexive closure by $\preccurlyeq$. We start with a multi-objective optimization problem which requires that the sub-problems be prioritized in lexicographical order [8,9,53,56,64].

**Definition 2 (Lexicographic Optimization (LO)).** *A lexicographic optimization problem is a sequence of GOMT problems* $\mathcal{LO} = (\mathcal{GO}_1, \ldots, \mathcal{GO}_n)$, *where* $\mathcal{GO}_i := \langle t_i, \prec_i, \phi_i \rangle$ *for* $i \in [1, n]$. *For* $\mathcal{T}$*-interpretations* $\mathcal{I}$ *and* $\mathcal{I}'$, *we say that:*

- $\mathcal{I}$ $\mathcal{LO}$*-dominates* $\mathcal{I}'$, *denoted by* $\mathcal{I} <_{\mathcal{LO}} \mathcal{I}'$, *if* $\mathcal{I}$ *and* $\mathcal{I}'$ *are* $\mathcal{GO}_i$-consistent *for each* $i \in [1, n]$, *and for some* $j \in [1, n]$*:*
  *(i)* $t_i^{\mathcal{I}} = t_i^{\mathcal{I}'}$ *for all* $i \in [1, j)$*; and*
  *(ii)* $t_j^{\mathcal{I}} \prec_j t_j^{\mathcal{I}'}$.
- $\mathcal{I}$ *is a solution to* $\mathcal{LO}$ *iff* $\mathcal{I}$ *is* $\mathcal{GO}_i$*-consistent for each* $i$ *and no* $\mathcal{T}$*-interpretation* $\mathcal{LO}$*-dominates* $\mathcal{I}$.

An $\mathcal{LO}$ problem can be solved by converting it into an instance of Definition 1.

**Definition 3 ($\mathcal{GO}_{\mathcal{LO}}$).** *Given an* $\mathcal{LO}$ *problem* $(\mathcal{GO}_1, \ldots, \mathcal{GO}_n)$, *with* $\mathcal{GO}_i := \langle t_i, \prec_i, \phi_i \rangle$ *for* $i \in [1, n]$, *the corresponding* $\mathcal{GO}$ *instance is defined as* $\mathcal{GO}_{\mathcal{LO}}(\mathcal{GO}_1, \ldots, \mathcal{GO}_n) := \langle t, \prec_{\mathcal{LO}}, \phi \rangle$, *where:*

- $t = tup(t_1, \ldots, t_n)$; $\quad \phi = \phi_1 \wedge \cdots \wedge \phi_n$;
- *if* $t$ *is of sort* $\sigma$, *then* $\prec_{\mathcal{LO}}$ *is the lexicographic extension of* $(\prec_1, \ldots, \prec_n)$ *to* $\sigma^{\mathcal{T}}$*: for* $(a_1, \ldots, a_n), (b_1, \ldots, b_n) \in \sigma^{\mathcal{T}}$, $(a_1, \ldots, a_n) \prec_{\mathcal{LO}} (b_1, \ldots, b_n)$ *iff for some* $j \in [1, n]$ *:*
  *(i)* $a_i = b_i$ *for all* $i \in [1, j)$*; and*
  *(ii)* $a_j \prec_j b_j$.

Here and in other definitions below, we use the data type theory constructor *tup* to construct the objective term $t$. This is a convenient mechanism for keeping an ordered list of the sub-objectives and keeps the overall theoretical framework simple. In practice, if using a solver that does not support tuples or the theory of data types, other implementation mechanisms could be used. Note that if each sub-problem uses a total order, then $\prec_{\mathcal{LO}}$ will also be total.

**Proposition 1.** *Let* $\mathcal{I}$ *be a* $\mathcal{GO}_{\mathcal{LO}}$*-solution. Then* $\mathcal{I}$ *is also a solution to the corresponding* $\mathcal{LO}$ *problem as defined in Definition 2.*

*Example 4 ($\mathcal{LO}$).* Let $\mathcal{GO}_1 := \langle x, \prec_1, True \rangle$ and $\mathcal{GO}_2 := \langle y +_{[2]} z, \prec_2, True \rangle$, where $x, y, z$ are variables of sort $BV_{[2]}$, $a \prec_1 b$ iff $a \prec_{[2]} b$, and $a \prec_2 b$ iff $a \succ_{[2]} b$. Now, let $\mathcal{GO} = \mathcal{GO}_{\mathcal{LO}}(\mathcal{GO}_1, \mathcal{GO}_2) = \langle t, \prec_{\mathcal{LO}}, True \rangle$. Then, $t = tup(x, y +_{[2]} z)$ and $(a_1, a_2) \prec_{\mathcal{LO}} (b_1, b_2)$ iff $a_1 \prec_{[2]} b_1$ or ($a_1 = b_1$ and $a_2 \succcurlyeq_{[2]} b_2$).
Now, let $\mathcal{I}, \mathcal{I}'$, and $\mathcal{I}''$ be such that: $x^{\mathcal{I}} = 11, y^{\mathcal{I}} = 00, z^{\mathcal{I}} = 10$, and $t^{\mathcal{I}} := (11, 10)$; $x^{\mathcal{I}'} = 01, y^{\mathcal{I}'} = 01, z^{\mathcal{I}'} = 01$, and $t^{\mathcal{I}'} := (01, 10)$; $x^{\mathcal{I}''} = 01, y^{\mathcal{I}''} = 01, z^{\mathcal{I}''} = 10$, and $t^{\mathcal{I}''} := (01, 11)$. Then, $\mathcal{I}'' <_{\mathcal{GO}} \mathcal{I}' <_{\mathcal{GO}} \mathcal{I}$, since $(01, 11) \prec_{\mathcal{LO}} (01, 10) \prec_{\mathcal{LO}} (11, 10)$.

We can also accommodate Pareto optimization [8,9,64] in our framework.

**Definition 4 (Pareto Optimization (PO)).** *A Pareto optimization problem is a sequence of GOMT problems* $\mathcal{PO} = (\mathcal{GO}_1, \ldots, \mathcal{GO}_n)$*, where* $\mathcal{GO}_i := \langle t_i, \prec_i, \phi_i \rangle$ *for* $i \in [1, n]$*. For any* $\mathcal{T}$*-interpretations* $\mathcal{I}$ *and* $\mathcal{I}'$*, we say that:*

- $\mathcal{I}$ *$\mathcal{PO}$-dominates, or Pareto dominates,* $\mathcal{I}'$*, denoted by* $\mathcal{I} <_{\mathcal{PO}} \mathcal{I}'$*, if* $\mathcal{I}$ *and* $\mathcal{I}'$ *are* $\mathcal{GO}$-consistent *w.r.t. each* $\mathcal{GO}_i$*,* $i \in [1, n]$*, and:*
  *(i)* $t_i^{\mathcal{I}} \preccurlyeq_i t_i^{\mathcal{I}'}$ *for all* $i \in [1, n]$*; and*
  *(ii) for some* $j \in [1, n]$*,* $t_j^{\mathcal{I}} \prec_j t_j^{\mathcal{I}'}$*.*
- $\mathcal{I}$ *is a solution to* $\mathcal{PO}$ *iff* $\mathcal{I}$ *is* $\mathcal{GO}$*-consistent w.r.t. each* $\mathcal{GO}_i$ *and no* $\mathcal{I}'$ $\mathcal{PO}$*-dominates* $\mathcal{I}$*.*

**Definition 5 ($\mathcal{GO}_{\mathcal{PO}}$).** *Given a PO problem* $\mathcal{PO} = (\mathcal{GO}_1, \ldots, \mathcal{GO}_n)$*, we define* $\mathcal{GO}_{\mathcal{PO}}(\mathcal{GO}_1, \ldots, \mathcal{GO}_n) := \langle t, \prec_{\mathcal{PO}}, \phi \rangle$*, where:*

- $t = tup(t_1, \ldots, t_n)$; $\quad \phi = \phi_1 \wedge \cdots \wedge \phi_n$;
- *if* $t$ *is of sort* $\sigma$*, then* $\prec_{\mathcal{PO}}$ *is the pointwise extension of* $(\prec_1, \ldots, \prec_n)$ *to* $\sigma^{\mathcal{T}}$*; for any* $(a_1, \ldots, a_n), (b_1, \ldots, b_n) \in \sigma^{\mathcal{T}}$*,* $(a_1, \ldots, a_n) \prec_{\mathcal{PO}} (b_1, \ldots, b_n)$ *iff:*
  *(i)* $a_i \preccurlyeq_i b_i$ *for all* $i \in [1, n]$*; and*
  *(ii)* $a_j \prec_j b_j$ *for some* $j \in [1, n]$*.*

**Proposition 2.** *Let* $\mathcal{I}$ *be a* $\mathcal{GO}_{\mathcal{PO}}$*-solution. Then* $\mathcal{I}$ *is also a solution to the corresponding* $\mathcal{PO}$ *problem as defined in Definition 4.*

Next, consider a $\mathcal{PO}$ example with two sub-problems: one minimizing the length of a string $w$, and the other maximizing a substring $x$ of $w$ lexicographically.

*Example 5 ($\mathcal{PO}$).* Let $\mathcal{T}$ be the SMT-LIB theory of strings and let $\mathcal{GO}_1 := \langle \mathsf{len}(w), \prec_1, \mathsf{len}(w) < 4 \rangle$ and $\mathcal{GO}_2 := \langle x, \prec_2, \mathsf{contains}(w, x) \rangle$, where $w$, $x$ are variables of sort $\mathsf{Str}$, $\prec_1$ is $\prec_{\mathsf{Int}}$, and $\prec_2$ is $\succ_{\mathsf{Str}}$. Now, let $\mathcal{GO}_{\mathcal{PO}} = \mathcal{GO}_{\mathcal{PO}}(\mathcal{GO}_1, \mathcal{GO}_2) = \langle t, \prec_{\mathcal{PO}}, \mathsf{len}(w) < 4 \wedge \mathsf{contains}(x, w) \rangle$. Then, $t = tup(\mathsf{len}(w), x)$ and $(a_1, a_2) \prec_{\mathcal{PO}} (b_1, b_2)$ iff $a_1 \preccurlyeq_{\mathsf{Int}} b_1$, $a_2 \succcurlyeq_{\mathsf{str}} b_2$, and ($a_1 \prec_{\mathsf{Int}} b_1$ or $a_2 \succ_{\mathsf{str}} b_2$). Now, let $\mathcal{I}$, $\mathcal{I}'$, and $\mathcal{I}''$ be such that: $\mathcal{I} := \{w \mapsto \texttt{"aba"}, x \mapsto \texttt{"ab"}\}$ and $t^{\mathcal{I}} := (3, \texttt{"ab"})$; $\mathcal{I}' := \{w \mapsto \texttt{"z"}, x \mapsto \texttt{"z"}\}$ and $t^{\mathcal{I}'} := (1, \texttt{"z"})$; and $\mathcal{I}'' := \{w \mapsto \epsilon, x \mapsto \epsilon\}$ and $t^{\mathcal{I}''} := (0, \epsilon)$. Then, $\mathcal{I}' <_{\mathcal{GO}} \mathcal{I}$, since $(1, \texttt{"z"}) \prec_{\mathcal{PO}} (3, \texttt{"ab"})$; but both $\mathcal{I}$ and $\mathcal{I}'$ are incomparable with $\mathcal{I}''$. Both $\mathcal{I}'$ and $\mathcal{I}''$ are optimal solutions.

Though we omit them for space reasons, we can similarly capture the MinMax and MaxMin optimization problems [56,64] as corresponding $\mathcal{GO}_{\mathcal{MIN}\mathcal{MAX}}$ and $\mathcal{GO}_{\mathcal{MAX}\mathcal{MIN}}$ instances of Definition 1.[3]

Note that except for degenerate cases, the orders used for MinMax and MaxMin, as well as the order $\prec_{\mathcal{PO}}$ above, are always partial orders. Being able to model these multi-objective optimization problems in a clean and simple way is a main motivation for using a partial instead of a total order in Definition 1.

Another problem in the literature is the *multiple-independent* (or *boxed*) optimization problem [8,9,64]. It simultaneously solves several independent GOMT problems. We show how to realize this as a single $\mathcal{GO}$ instance.

[3] Details of these formulations can be found in the longer version of this paper [67].

**Definition 6 (Boxed Optimization (BO)).** *A boxed optimization problem is a sequence of GOMT problems, $\mathcal{BO} = (\mathcal{GO}_1, \ldots, \mathcal{GO}_n)$, where $\mathcal{GO}_i := \langle t_i, \prec_i, \phi_i \rangle$ for $i \in [1, n]$. We say that:*

- *A sequence of interpretations $(\mathcal{I}_1, \ldots, \mathcal{I}_n)$ $\mathcal{BO}$-dominates $(\mathcal{I}'_1, \ldots, \mathcal{I}'_n)$, denoted by $(\mathcal{I}_1, \ldots, \mathcal{I}_n) <_{\mathcal{BO}} (\mathcal{I}'_1, \ldots, \mathcal{I}'_n)$, if $\mathcal{I}_i$ and $\mathcal{I}'_i$ are $\mathcal{GO}_i$-consistent for each $i \in [1, n]$, and:*
  *(i) $t_i^{\mathcal{I}_i} \preccurlyeq_i t_i^{\mathcal{I}'_i}$ for all $i \in [1, n]$; and*
  *(ii) for some $j \in [1, n]$, $t_j^{\mathcal{I}_j} \prec_j t_j^{\mathcal{I}'_j}$.*
- *$(\mathcal{I}_1, \ldots, \mathcal{I}_n)$ is a solution to $\mathcal{BO}$ iff $\mathcal{I}_i$ is $\mathcal{GO}_i$-consistent for each $i \in [1, n]$ and no $(\mathcal{I}'_1, \ldots, \mathcal{I}'_n)$ $\mathcal{BO}$-dominates $(\mathcal{I}_1, \ldots, \mathcal{I}_n)$.*

Note that in previous work, there is an additional assumption that $\phi_i = \phi_j$ for all $i, j \in [1, n]$. Below, we show how to solve the more general case without this assumption. We first observe that the above definition closely resembles Definition 4 for Pareto optimization (PO) problems. Leveraging this similarity, we show how to transform an instance of a BO problem into a PO problem.

**Definition 7.** *($\mathcal{GO}_{\mathcal{BO}}$) Let $\mathcal{BO} = (\mathcal{GO}_1, \ldots, \mathcal{GO}_n)$, where $\mathcal{GO}_i := \langle t_i, \prec_i, \phi_i \rangle$ for $i \in [1, n]$. Let $V_i$ be the set of all free variables in the $i^{th}$ sub-problem that also appear in at least one other sub-problem:*

$$V_i = (FV(t_i) \cup FV(\phi_i)) \cap \bigcup_{j \in [1,n], j \neq i} FV(t_j) \cup FV(\phi_j).$$

*Let $\boldsymbol{v_i} = (v_{i,1}, \ldots, v_{i,m})$ be some ordering of the variables in $V_i$ (say, by $\prec_{\mathcal{X}}$), and for each $j \in [1, m]$, let $v'_{i,j}$ be a fresh variable of the same sort as $v_{i,j}$, and let $\boldsymbol{v'_i} = (v'_{i,1}, \ldots, v'_{i,m})$. Then, let $t'_i = t_i[\boldsymbol{v_i} \leftarrow \boldsymbol{v'_i}]$, $\phi'_i = \phi_i[\boldsymbol{v_i} \leftarrow \boldsymbol{v'_i}]$, and $\mathcal{GO}'_i = \langle t'_i, \prec_i, \phi'_i \rangle$. Then we define $\mathcal{GO}_{\mathcal{BO}} := \mathcal{GO}_{\mathcal{PO}}(\mathcal{GO}'_1, \ldots, \mathcal{GO}'_n)$.*

**Proposition 3.** *Let $\mathcal{I}$ be a solution to $\mathcal{GO}_{\mathcal{BO}}$ as defined in Definition 7. Then $(\mathcal{I}_1, \ldots, \mathcal{I}_n)$ is a solution to the corresponding $\mathcal{BO}$ problem as defined in Definition 6, where for each $i \in [1, n]$, $\mathcal{I}_i$ is the same as $\mathcal{I}$ except that each variable $v_{i,j} \in V_i$ is interpreted as $(v'_{i,j})^{\mathcal{I}}$.*

In practice, solvers for BO problems can be implemented without variable renaming (see, e.g., [8,36,53]). Variable renaming, while a useful theoretical construct, also adds generality to our definition of $\mathcal{BO}$. An interesting direction for future experimental work would be to compare the two approaches in practice.

**Compositional Optimization.** GOMT problems can also be combined by functional composition of multiple objective terms, possibly of different sorts, yielding *compositional optimization problems* [12,62,64]. Our framework handles them naturally by simply constructing an objective term capturing the desired compositional relationship. For example, compositional objectives can address the (partial) MaxSMT problem [64], where some formulas are *hard* constraints and others are *soft* constraints. The goal is to satisfy all hard constraints and as many soft constraints as possible. The next example is inspired by Cimatti et al. [12] and Teso et al. [62].

*Example 6 (*MaxSMT*).* Let $x \geq 0$ and $y \geq 0$ be hard constraints and $4x+y-4 \geq 0$ and $2x + 3y - 6 \geq 0$ soft constraints. We can formalize this as $\mathcal{GO}_{\mathcal{CO}} = \langle t, \prec, \phi \rangle$, where: $t = ite(4x + y - 4 \geq 0, 0, 1) + ite(2x + 3y - 6 \geq 0, 0, 1)$, $\prec \equiv \prec_{\mathsf{Int}}$, and $\phi = x \geq 0 \wedge y \geq 0$. An optimal solution must satisfy both hard constraints and, by minimizing the objective term $t$, as many soft constraints as possible.

MaxSMT has various variants including generalized, partial, weighted, and partial weighted MaxSMT [64], all of which our framework can handle similarly.

Next, we show a different compositional example that combines two different orders, one on strings and the other on integers. This example also illustrates a theory combination not present in the OMT literature.

*Example 7 (Composition of Str and Int).* Let $\mathcal{T}$ be again the theory of strings[4] Let $\mathcal{GO}_{\mathcal{CO}} = \langle tup(x, \mathsf{len}(x)), \prec, \mathsf{contains}(x, \texttt{"a"}) \wedge \mathsf{len}(x) > 1 \rangle$, where $x$ is of sort Str and $(a_1, b_1) \prec (a_2, b_2)$ iff $b_1 \prec_{\mathsf{Int}} b_2$ or $(b_1 = b_2$ and $a_1 \succ_{\mathsf{str}} a_2)$. $\prec$ prioritizes minimizing the length, but then maximizes the string with respect to lexicographic order. An optimal solution must interpret $x$ as the string `"za"` of length 2 since $x$ must be of length at least 2 and contain `"a"`, making `"za"` the largest string of minimum length.

Based on the definitions given in this section, we see that our formalism can capture any combination of $\mathcal{GO}$ (including compositional), $\mathcal{GO}_{\mathcal{LO}}$, $\mathcal{GO}_{\mathcal{PO}}$, $\mathcal{GO}_{\mathcal{MIN}\mathcal{MAX}}$, $\mathcal{GO}_{\mathcal{MAX}\mathcal{MIN}}$, and $\mathcal{GO}_{\mathcal{BO}}$ problems. And note that the last four all make use of the partial order feature of Definition 1.

## 4 The GOMT Calculus

We introduce a calculus for solving the GOMT problem, presented as a set of *derivation rules*. We fix a GOMT problem $\mathcal{GO} = \langle t, \prec, \phi \rangle$ where $\phi$ is satisfiable (optimizing does not make sense otherwise). We start with a few definitions.

**Definition 8 (State).** *A* state *is a tuple $\Psi = \langle \mathcal{I}, \Delta, \tau \rangle$, where $\mathcal{I}$ is an interpretation, $\Delta$ is a formula, and $\tau$ is a sequence of formulas.*

The set of all states forms the state space for the GOMT problem. Intuitively, the proof procedure of the calculus is a search procedure over this state space which maintains at all times a *current state* $\langle \mathcal{I}, \Delta, \tau \rangle$ storing a candidate solution and additional search information. In the current state, $\mathcal{I}$ is the best solution found so far in the search; $\Delta$ is a formula describing the remaining, yet unexplored, part of the state space, where a better solution might exist; and $\tau$ contains formulas that divide up the search space described by $\Delta$ into *branches* represented by the individual formulas in $\tau$, maintaining the invariant that the disjunction of all the formulas $\tau_1, \ldots, \tau_p$ in $\tau$ is equivalent to $\Delta$ modulo $\phi$, that is, $\phi \models (\bigvee_{i=1}^{p} \tau_i \Leftrightarrow \Delta)$.

Note that states contain $\mathcal{T}$-interpretations, which are possibly infinite mathematical structures. This is useful to keep the calculus simple. In practice, it

[4] The SMT-LIB theory of strings includes the theory of integers to support constraints over string length.

is enough just to keep track of the interpretations of the (finitely-many) symbols without fixed meanings (variables and uninterpreted functions and sorts) appearing in the state, much as SMT solvers do in order to produce models.

**Definition 9 (Solve).** SOLVE *is a function that takes a formula and returns a satisfying interpretation if the formula is satisfiable and a distinguished value* $\perp$ *otherwise.*

**Definition 10 (Better).** $\text{BETTER}_{\mathcal{GO}}$ *is a function that takes a* $\mathcal{GO}$*-consistent interpretation* $\mathcal{I}$ *and returns a formula* $\text{BETTER}_{\mathcal{GO}}(\mathcal{I})$ *with the property that for every* $\mathcal{GO}$*-consistent interpretation* $\mathcal{I}'$*,*

$$\mathcal{I}' \models \text{BETTER}_{\mathcal{GO}}(\mathcal{I}) \quad \textit{iff } \mathcal{I}' <_{\mathcal{GO}} \mathcal{I}.$$

The function above is specific to the given optimization problem $\mathcal{GO}$ or, put differently, is parametrized by $t$, $\prec$, and $\phi$. When $\mathcal{GO}$ is clear, however, we simply write BETTER, for conciseness.

The calculus relies on the existence and computability of SOLVE and BETTER. SOLVE can be realized by any standard SMT solver. BETTER relies on a defining formula for $\prec$ as discussed below. We note that intuitively, $\text{BETTER}(\mathcal{I})$ is simply a (possibly unsatisfiable) formula characterizing the solutions of $\phi$ that are better than $\mathcal{I}$. Assuming $\alpha_\prec$ is the formula defining $\prec$, with free variables $v_1 \prec_{\mathcal{X}} v_2$, if the value $t^{\mathcal{I}}$ can be represented by some constant $c$ (e.g., if $t^{\mathcal{I}}$ is a rational number), then $\text{BETTER}(\mathcal{I}) = \alpha_\prec[(v_1, v_2) \leftarrow (t, c)]$ satisfies Definition 10. On the other hand, it could be that $t^{\mathcal{I}}$ is not representable as a constant (e.g., it could be an algebraic real number); then, a more sophisticated formula (involving, say, a polynomial and an interval specifying a particular root) may be required.

**Definition 11 (Initial State).** *The* initial state *of the GOMT problem* $\mathcal{GO} = \langle t, \prec, \phi \rangle$ *is* $\langle \mathcal{I}_0, \Delta_0, \tau_0 \rangle$*, where* $\mathcal{I}_0 = \text{SOLVE}(\phi)$*,* $\Delta_0 = \text{BETTER}(\mathcal{I}_0)$*,* $\tau_0 = (\Delta_0)$*.*

Note that $\mathcal{I}_0 \neq \perp$ since we assume that $\phi$ is satisfiable. The search for an optimal solution to the GOMT problem in our calculus starts with an arbitrary solution of the constraint $\phi$ and continues until it finds an optimal one.

### 4.1 Derivation Rules

Figure 1 presents the derivation rules of the GOMT calculus. The rules are given in guarded assignment form, where the rule premises describe the conditions on the current state that must hold for the rule to apply, and the conclusion describes the resulting modifications to the state. State components not mentioned in the conclusion of a rule are unchanged.

A derivation rule *applies* to a state if (*i*) the conditions in the premise are satisfied by the state and (*ii*) the resulting state is different. A state is *saturated* if no rules apply to it. A $\mathcal{GO}$*-derivation* is a sequence of states, possibly infinite, where the first state is the initial state of the GOMT problem $\mathcal{GO}$, and each state

$$\text{F-Split}\ \frac{\tau \neq \emptyset \quad \psi = \textsc{Top}(\tau) \quad \phi \models \psi \Leftrightarrow \bigvee_{j=1}^{k} \psi_j,\ k \geq 1}{\tau := (\psi_1, \ldots, \psi_k) \circ \textsc{Pop}(\tau)}$$

$$\text{F-Sat}\ \frac{\tau \neq \emptyset \quad \psi = \textsc{Top}(\tau) \quad \textsc{Solve}(\phi \wedge \psi) = \mathcal{I}' \quad \mathcal{I}' \neq \bot \quad \Delta' = \Delta \wedge \textsc{Better}(\mathcal{I}')}{\mathcal{I} := \mathcal{I}',\ \Delta := \Delta',\ \tau := (\Delta')}$$

$$\text{F-Close}\ \frac{\tau \neq \emptyset \quad \psi = \textsc{Top}(\tau) \quad \textsc{Solve}(\phi \wedge \psi) = \bot}{\Delta := \Delta \wedge \neg\psi,\ \tau := \textsc{Pop}(\tau)}$$

**Fig. 1.** The derivation rules of the GOMT Calculus.

in the sequence is obtained by applying one of the rules to the previous state. The *solution sequence* of a derivation is the sequence made up of the solutions (i.e., the interpretations) in each state of the derivation.

The calculus starts with a solution for $\phi$ and improves on it until an optimal solution is found. During a derivation, the best solution found so far is maintained in the $\mathcal{I}$ component of the current state. A search for a better solution can be organized into branches through the use of the F-Split rule. Progress toward a better solution is enforced by the formula $\Delta$ which, by construction, is falsified by all the solutions found so far. We elaborate on the individual rules next.

**F-Split.** F-Split divides the branch of the search space represented by the top formula $\psi = \textsc{Top}(\tau)$ in $\tau$ into $k$ sub-branches $(\psi_1, \ldots, \psi_k)$, ensuring their disjunction is equivalent to $\psi$ modulo the constraint $\phi$: $\phi \models \psi \Leftrightarrow \bigvee_{j=1}^{k} \psi_j$. The rest of the state remains unchanged. F-Split is applicable whenever $\tau$ is non-empty. The rule does not specify how the formulas $\psi_1, \ldots, \psi_k$ are chosen. However, a pragmatic implementation should aim to generate them so that they are *irredundant* in the sense that no formula is entailed modulo $\phi$ by the (disjunction of the) other formulas. This way, each branch potentially contains a solution that the others do not. Note, however, that this is not a requirement.

**F-Sat.** The F-Sat rule applies when there is a solution in the branch represented by the top formula $\psi$ in $\tau$. The rule selects a solution $\mathcal{I}' = \textsc{Solve}(\phi \wedge \psi)$ from that branch. One can prove that, by the way the formulas in $\tau$ are generated in the calculus, $\mathcal{I}'$ necessarily improves on the current solution $\mathcal{I}$, moving the search closer to an optimal solution.[5] Thus, F-Sat switches to the new solution (with $\mathcal{I} := \mathcal{I}'$) and directs the search to seek an even better solution by updating $\Delta$ to $\Delta' = \Delta \ \wedge\ \textsc{Better}(\mathcal{I}')$. Note that F-Sat resets $\tau$ to the singleton sequence $(\Delta')$, discarding any formulas in $\tau$. This is justified, as any discarded better solutions must also be in the space defined by $\Delta'$.

**F-Close.** The F-Close rule eliminates the first element $\psi$ of a non-empty $\tau$ if the corresponding branch contains no solutions (i.e., $\textsc{Solve}(\phi \wedge \psi) = \bot$).

[5] See Lemma 6 in Appendix B of a longer version of this paper [67].

The rule further updates the state by adding the negation of $\psi$ to $\Delta$ as a way to eliminate from further consideration the interpretations satisfying $\psi$.

Note that rules F-Sat and F-Close both update $\Delta$ to reflect the remaining search space, whereas F-Split refines the division of the current search space.

### 4.2 Search Strategies

The GOMT calculus provides the flexibility to support different search strategies. Here, we give some examples, including both notable strategies from the OMT literature as well as new strategies enabled by the calculus, and explain how they work at a conceptual level.

**Divergence of Strategies:** The strategies discussed below, with the exception of Hybrid search, may diverge if an optimal solution does not exist or if there is a *Zeno-style* [54,55] infinite chain of increasingly better solutions, all dominated by an optimal one. We discuss these issues and termination in general in Sect. 4.4.

**Linear Search:** A linear search strategy is obtained by never using the F-Split rule. Instead, the F-Sat rule is applied to completion (that is, repeatedly until it no longer applies). As we show later (see Theorem 2), in the absence of Zeno chains, $\tau$ eventually becomes empty, terminating the search. At that point, $\mathcal{I}$ is guaranteed to be an optimal solution.

**Binary Search:** A binary search strategy is achieved by using the F-Split rule to split the search space represented by $\psi = \textsc{Top}(\tau)$ into two subspaces, represented by two formulas $\psi_1$ and $\psi_2$, with $\phi \models \psi \Leftrightarrow (\psi_1 \vee \psi_2)$. In a strict binary search strategy, $\psi_1$ and $\psi_2$ should be chosen so that the two subspaces are disjoint and, to the extent possible, of equal size. A typical binary strategy alternates applications of F-Split with applications of either F-Sat or F-Close until $\tau$ becomes empty, at which point $\mathcal{I}$ is guaranteed to be an optimal solution. A smart strategy would aim to find an optimal solution as soon as possible by arranging for solutions in $\psi_1$ (which will be checked first) to be better than solutions in $\psi_2$, if this is easy to determine. Note that an unfortunate choice of $\psi_1$ by F-Split, containing no solutions at all, is quickly remedied by an application of F-Close which removes $\psi_1$, allowing $\psi_2$ to be considered next. The same problem of Zeno-style infinite chains can occur in this strategy.

**Multi-directional Exploration:** For multi-objective optimization problems, a search strategy can be defined to simultaneously direct the search space towards any or all objectives. Formally, if $n$ is the number of objectives, then the F-Split rule can be instantiated in such a way that $\psi_j = \bigwedge_{i=1}^{n} \psi_{ji}$, where $\psi_{ji}$ is a formula describing a part of the search space for the $i^{th}$ objective term in the $j^{th}$ branch.

**Search Order:** We formalize $\tau$ as a sequence to enforce exploring the branches in $\tau$ in a specific order, assuming such an order can be determined at the time of applying F-Split. Often, this is the case. For example, in binary search, it is typically best to explore the section of the search space with better objective values first. If a solution is found in this section, a larger portion of the search space is pruned. Conversely, if the branches are explored in another order, even finding a solution necessitates continued exploration of the space corresponding to the remaining branches.

Alternatively, $\tau$ can be implemented as a set, by redefining the Top and Pop functions accordingly to select and remove a desired element in $\tau$. With $\tau$ defined as a set, additional search strategies are possibile, including parallel exploration of the search space and the ability to arbitrarily switch between branches.

**Hybrid Search:** For some objectives and orders, there exist off-the-shelf external optimization procedures (e.g., Simplex for linear real arithmetic). One way to integrate such a procedure into our calculus is to replace a call to the Solve function in F-Sat with a call to an external optimization procedure Optimize that is sort- and order-compatible with the GOMT problem. We pass to Optimize as parameters the constraint $\phi \wedge \textsc{Top}(\tau)$ and the objective $t$ and obtain an optimal solution in the current branch $\textsc{Top}(\tau)$.[6] The call can be viewed as an accelerator for a linear search on the current branch. This approach incorporates theory-specific optimization solvers in much the same way as is done in the OMT literature. However, our calculus extends previous approaches with the ability to blend theory-specific optimization with theory-agnostic optimization by interleaving applications of F-Sat using Solve with applications using Optimize. For example, we may want to alternate between expensive calls to an external optimization solver and calls to a standard solver that are guided by a custom branching heuristic.

**Other Strategies:** The calculus enables us to mix and match the above strategies arbitrarily, as well as to model other notable search techniques like cutting planes [16] by integrating a cut formula into Solve. And, of course, one advantage of an abstract calculus is that its generality provides a framework for the exploration of new strategies. Such an exploration is a promising direction for future work.

### 4.3 New Applications

A key feature of our framework is that it is theory-agnostic, that is, it can be used with any SMT theory or combination of theories. This is in contrast to most of the OMT literature in which a specific theory is targeted. It also fully supports arbitrary composition of GOMT problems using the multi-objective approaches described in Sect. 3.2. Thus, our framework enables OMT to be extended to new

[6] This assumes there exists an optimal solution in the current branch. If not (i.e., if the problem is unbounded), a suitable error can be raised and the search terminated.

application areas requiring either combinations of theories or multi-objective formulations that are unsupported by previous approaches. We illustrate this (and the calculus itself) using a Pareto optimization problem over the theories of strings and integers (a combination of theories and objectives unsupported by any existing OMT approach or solver).

*Example 8 ($\mathcal{GO}_{\mathcal{PO}}$).* Let $\mathcal{GO}_1 := \langle \mathsf{len}(w), \prec_1, \mathsf{len}(s) < \mathsf{len}(w) \rangle$ and $\mathcal{GO}_2 := \langle x, \prec_2, x = s \cdot w \cdot s \rangle$, where $w$, $x$, $s$ are of sort $\mathsf{Str}$, $\mathsf{len}(w)$ and $\mathsf{len}(s)$ are of sort $\mathsf{Int}$, $\prec_1 \equiv \prec_{\mathsf{Int}}$, and $\prec_2 \equiv \succ_{\mathsf{Str}};$. Then, let $\mathcal{GO}_{\mathcal{PO}}(\mathcal{GO}_1, \mathcal{GO}_2) := \langle t, \prec_{\mathcal{PO}}, \phi \rangle$, where $t$ is $tup(\mathsf{len}(w), x)$, $\phi$ is $x = s \cdot w \cdot s \ \wedge \ \mathsf{len}(s) < \mathsf{len}(w)$, and $(a_1, a_2) \prec_{\mathcal{PO}} (b_1, b_2)$ iff $a_1 \preccurlyeq_1 b_1$, $a_2 \preccurlyeq_2 b_2$, and either $a_1 \prec_1 b_1$ or $a_2 \prec_2 b_2$ or both. Suppose initially:

$$\begin{aligned} \mathcal{I}_0 &= \{x \mapsto \texttt{"aabaa"}, s \mapsto \texttt{"a"}, w \mapsto \texttt{"aba"}, \}, \quad \tau_0 = \ (\Delta_0), \\ \Delta_0 &= (\mathsf{len}(w) \leq 3 \wedge x >_{\mathsf{str}} \texttt{"aabaa"}) \vee (\mathsf{len}(w) < 3 \wedge x \geq_{\mathsf{str}} \texttt{"aabaa"}). \end{aligned}$$

The initial objective term value is $(3, \texttt{"aabaa"})$.

1. We can first apply F-Split to split the top-level disjunction in $\tau$. And suppose we want to work on the second disjunct first. This results in:

$$\tau_1 = (\mathsf{len}(w) < 3 \wedge x \geq_{\mathsf{str}} \texttt{"aabaa"}, \mathsf{len}(w) \leq 3 \wedge x >_{\mathsf{str}} \texttt{"aabaa"})$$

   while the other elements of the state are unchanged.
2. Now, suppose we want to do binary search on the length objective. This can be done by again applying the F-Split rule with the disjunction $(\mathsf{len}(w) < 2 \wedge x \geq_{\mathsf{str}} \texttt{"aabaa"}) \vee (2 \leq \mathsf{len}(w) < 3 \wedge x \geq_{\mathsf{str}} \texttt{"aabaa"})$ to get:

$$\begin{aligned} \tau_2 = \ & (\mathsf{len}(w) < 2 \wedge x \geq_{\mathsf{str}} \texttt{"aabaa"}, 2 \leq \mathsf{len}(w) < 3 \wedge x \geq_{\mathsf{str}} \texttt{"aabaa"}, \\ & \mathsf{len}(w) \leq 3 \wedge x >_{\mathsf{str}} \texttt{"aabaa"}). \end{aligned}$$

3. Both F-Split and F-Sat are applicable, but we follow the strategy of applying F-Sat after a split. Suppose we get the new solution $\mathcal{I}' = \{x \mapsto \texttt{"b"}, s \mapsto \epsilon, w \mapsto \texttt{"b"}\}$. Then we have:

$$\begin{aligned} \mathcal{I}_3 = \ & \{x \mapsto \texttt{"b"}, s \mapsto \epsilon, w \mapsto \texttt{"b"}\}, \quad \tau_3 = \ (\Delta_3), \\ \Delta_3 = \ & (\mathsf{len}(w) \leq 1 \wedge x >_{\mathsf{str}} \texttt{"b"}) \vee (\mathsf{len}(w) < 1 \wedge x \geq_{\mathsf{str}} \texttt{"b"}). \end{aligned}$$

4. Both F-Split and F-Sat are again applicable. Suppose that we switch now to linear search and thus again apply F-Sat, and suppose the new solution is $\mathcal{I}' = \{x \mapsto \texttt{"z"}, s \mapsto \epsilon, w \mapsto \texttt{"z"}\}$. This brings us to the state:

$$\begin{aligned} \mathcal{I}_4 = \ & \{x \mapsto \texttt{"z"}, s \mapsto \epsilon, w \mapsto \texttt{"z"}\}, \quad \tau_4 = \ (\Delta_4), \\ \Delta_4 = \ & (\mathsf{len}(w) \leq 1 \wedge x >_{\mathsf{str}} \texttt{"z"}) \vee (\mathsf{len}(w) < 1 \wedge x \geq_{\mathsf{str}} \texttt{"z"}). \end{aligned}$$

5. Now, $\textsc{Solve}(\phi \wedge ((\mathsf{len}(w) \leq 1 \wedge x >_{\mathsf{str}} \texttt{"z"}) \vee (\mathsf{len}(w) < 1 \wedge x \geq_{\mathsf{str}} \texttt{"z"}))) = \bot$. Indeed, $\mathsf{len}(w) \neq 0$, since $0 \leq \mathsf{len}(s) < \mathsf{len}(w)$; if $\mathsf{len}(w) = 1$, then $\mathsf{len}(s) = 0$ and $\mathsf{len}(x) = 1$, thus, $x \not>_{\mathsf{str}} \texttt{"z"}$. Now F-Close can derive the state:

$$\langle \mathcal{I}_5, \Delta_5, \tau_5 \rangle = \langle \mathcal{I}_4, \Delta_4 \wedge \neg \Delta_4, \emptyset \rangle$$

6. We have reached a saturated state, and $\mathcal{I}_5$ is a Pareto optimal solution. □

Optimization of objectives involving strings and integers (or strings and bitvectors) could be especially useful in security applications such as those mentioned in [60]. Optimization could be used in such applications to ensure that a counter-example is as simple as possible, for example.

Examples of multi-objective problems unsupported by existing solvers include multiple Pareto problems with a single min/max query, Pareto-lexicographic multi-objective optimization, and single Pareto queries involving MinMax and MaxMin optimization (see, for example, [1,32,52]). Our framework offers immediate solutions to these problems.

As has repeatedly been the case in SMT research, when new capabilities are introduced, new applications emerge. We expect that will happen also for the new capabilities introduced in this paper. One possible application is the optimization of emerging technology circuit designs [22].

### 4.4 Correctness

In this section, we establish correctness properties for $\mathcal{GO}$-derivations. Initially, we demonstrate that upon reaching a saturated state, the interpretation $\mathcal{I}$ in that state is optimal.[7]

**Theorem 1.** *(Solution Soundness) Let $\langle \mathcal{I}, \Delta, \tau \rangle$ be a saturated state in a derivation for a GOMT problem $\mathcal{GO}$. Then, $\mathcal{I}$ is an optimal solution to $\mathcal{GO}$.*

*Proof.* (Sketch) We show that in a saturated state $\tau = \emptyset$, and when $\tau = \emptyset$, $\phi \models \neg\Delta$. Then, we establish that $\mathcal{I}$ is $\mathcal{GO}$-consistent, and that for any $\mathcal{GO}$-consistent $\mathcal{T}$-interpretation $\mathcal{J}$, $\mathcal{J} \models \Delta$ iff $\mathcal{J} <_{\mathcal{GO}} \mathcal{I}$. This implies there is no $\mathcal{J}$ s.t. $\mathcal{J} \models \phi$ and $\mathcal{J} <_{\mathcal{GO}} \mathcal{I}$, confirming $\mathcal{I}$ as an optimal solution to $\mathcal{GO}$. □

In general, the calculus does not always have complete derivation strategies, for a variety of reasons. It could be that the problem is unbounded, i.e., no optimal solutions exist along some branch. Another possibility is that the order is not well-founded, and thus, an infinite sequence of improving solutions can be generated without ever reaching an optimal solution. For the former, various checks for unboundedness can be used. These are beyond the scope of this work, but some approaches are discussed in Trentin [64]. The latter can be overcome using a hybrid strategy when an optimization procedure exists (see Theorem 4). It is also worth observing that any derivation strategy is in effect an *anytime procedure*: forcibly stopping a derivation at any point yields (in the final state) the best solution found so far. When an optimal solution exists and is unique, stopping early provides the best approximation up to that point of the optimal solution.

There are also fairly general conditions under which solution complete derivation strategies do exist. We present them next.

[7] Full proofs for the theorems in this section can be found in a longer version of this paper [67].

**Definition 12.** *A derivation strategy is* progressive *if it (i) never halts in a non-saturated state and (ii) only uses* F-Split *a finite number of times in any derivation.*

Let us again fix a GOMT problem $\mathcal{GO} = \langle t, \prec, \phi \rangle$. Consider the set $A_t = \{t^{\mathcal{I}} \mid \mathcal{I} \text{ is } \mathcal{GO}\text{-consistent}\}$, collecting all values of $t$ in interpretations satisfying $\phi$.

**Theorem 2.** *(Termination) If $\prec$ is well-founded over $A_t$, any progressive strategy reaches a saturated state.*

*Proof.* (Sketch) We show that any derivation using a progressive strategy terminates when $\prec$ is well-founded. Subsequently, based on the definition of progressive, the final state must be saturated. □

**Theorem 3.** *(Solution Completeness) If $\prec$ is well-founded over $A_t$ and $\mathcal{GO}$ has one or more optimal solutions, every derivation generated by a progressive derivation strategy ends with a saturated state containing one of them.*

*Proof.* The proof is a direct consequence of Theorem 1 and Theorem 2. □

Solution completeness can also be achieved using an appropriate hybrid strategy.

**Theorem 4.** *If $\mathcal{GO}$ has one or more optimal solutions and is not unbounded along any branch, then every derivation generated by a progressive hybrid strategy, where* Solve *is replaced by* Optimize *in* F-Sat*, ends with a saturated state containing one of them.*

*Proof.* (Sketch) If $D$ is such a derivation, we note that F-Split can only be applied a finite number of times in $D$ and consider the suffix of $D$ after the last application of F-Split. In that suffix, F-Close can only be applied a finite number of times in a row, after which F-Sat must be applied. We then show that due to the properties of Optimize, this must be followed by either an application of F-Close or a single application of F-Sat followed by F-Close. Both cases result in saturated states. The theorem then follows from Theorem 1. □

## 5 Related Work

Various approaches for solving the OMT problem have been proposed. We summarize the key ideas below and refer the reader to Trentin [64] for a more thorough survey.

The *offline schema* employs an SMT solver as a black box for optimization search through incremental calls [54,55], following linear- or binary-search strategies. Initial bounds on the objective function are given and iteratively tightened after each call to the SMT solver. In contrast, the *inline schema* conducts the optimization search within the SMT solver itself [54,55], integrating the optimization criteria into its internal algorithm. While the inline schema can be more efficient than the offline counterpart, it necessitates invasive changes to the solver and may not be possible for every theory.

*Symbolic Optimization* optimizes multiple independent linear arithmetic objectives simultaneously [36], seeking optimal solutions for each corresponding objective. This approach improves performance by sharing SMT search effort. It exists in both offline and inline versions, with the latter demonstrating superior performance. Other arithmetic schemas combine simplex, branch-and-bound, and cutting-plane techniques within SMT solvers [44,50]. A polynomial constraint extension has also been introduced [33].

Theory-specific techniques address objectives involving pseudo-Booleans [11, 54,55,57], bitvectors [40,65], bitvectors combined with floating-point arithmetic [66], and nonlinear arithmetic [6]. Other related work includes techniques for lexicographic optimization [8], Pareto optimization [8,24], MaxSMT [19], and All-OMT [64].

Our calculus is designed to capture all of these variations. It directly corresponds to the offline schema, can handle both single- and multi-objective problems, and can integrate solvers with inline capabilities (including theory-specific ones) using the hybrid solving strategy. Efficient MaxSMT approaches [19] can also be mimicked in our calculus. These approaches systematically explore the search space by iteratively processing segments derived from unsat cores. Our calculus can instantiate these branches using the F-Split rule, by first capturing unsat cores from calls to F-Close, and then using these cores to direct the search in the F-Split rule.

## 6 Conclusion and Future Work

This paper introduces the Generalized OMT problem, a proper extension of the OMT problem. It also provides a general setting for formalizing various approaches for solving the problem in terms of a novel calculus for GOMT and proves its key correctness properties. As with previous work on abstract transition systems for SMT [27,31,39,45], this work establishes a framework for both theoretical exploration and practical implementations. The framework is general in several aspects: (*i*) it is parameterized by the optimization order, which does not need to be total; (*ii*) it unifies single- and multi-objective optimization problems in a single definition; (*iii*) it is theory-agnostic, making it applicable to any theory or combination of theories; and (*iv*) it provides a formal basis for understanding and exploring search strategies for Generalized OMT.

In future work, we plan to explore an extension of the calculus to the generalized All-OMT problem. We also plan to develop a concrete implementation of the calculus in a state-of-the-art SMT solver and evaluate it experimentally against current OMT solvers.

**Acknowledgements.** This work was funded in part by the Stanford Agile Hardware Center and by the National Science Foundation (grant 2006407).

## References


1. Akinlana, D.M.: New Developments in Statistical Optimal Designs for Physical and Computer Experiments. Ph.D. thesis, University of South Florida (2022)
2. Barrett, C., Fontaine, P., Tinelli, C.: The Satisfiability Modulo Theories Library (SMT-LIB) (2016). www.SMT-LIB.org
3. Barrett, C.W., Sebastiani, R., Seshia, S.A., Tinelli, C.: Satisfiability modulo theories. In: Handbook of Satisfiability, Frontiers in Artificial Intelligence and Applications, vol.185, pp. 825–885. IOS Press (2009)
4. Bertolissi, C., dos Santos, D.R., Ranise, S.: Solving multi-objective workflow satisfiability problems with optimization modulo theories techniques. In: Bertino, E., Lin, D., Lobo, J. (eds.) Proceedings of the 23nd ACM on Symposium on Access Control Models and Technologies, SACMAT 2018, Indianapolis, IN, USA, June 13-15, 2018, pp. 117–128. ACM (2018)
5. Bian, Z., Chudak, F., Macready, W., Roy, A., Sebastiani, R., Varotti, S.: Solving SAT and MaxSAT with a quantum annealer: foundations and a preliminary report. In: Dixon, C., Finger, M. (eds.) FroCoS 2017. LNCS (LNAI), vol. 10483, pp. 153–171. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-66167-4_9
6. Bigarella, F., et al.: Optimization modulo non-linear arithmetic via incremental linearization. In: Konev, B., Reger, G. (eds.) FroCoS 2021. LNCS (LNAI), vol. 12941, pp. 213–231. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-86205-3_12
7. Bit-Monnot, A., Leofante, F., Pulina, L., Ábrahám, E., Tacchella, A.: SMarTplan: a task planner for smart factories (2018). arXiv preprint arXiv:1806.07135
8. Bjørner, N.S., Phan, A.D.: $\nu$z - maximal satisfaction with z3. In: International Symposium on Symbolic Computation in Software Science (2014)
9. Bjørner, N., Phan, A.-D., Fleckenstein, L.: $\nu$Z - An optimizing SMT solver. In: Baier, C., Tinelli, C. (eds.) TACAS 2015. LNCS, vol. 9035, pp. 194–199. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46681-0_14
10. Candeago, L., Larraz, D., Oliveras, A., Rodríguez-Carbonell, E., Rubio, A.: Speeding up the constraint-based method in difference logic. In: Creignou, N., Le Berre, D. (eds.) Theory and Applications of Satisfiability Testing - SAT 2016, pp. 284–301. Springer International Publishing, Cham (2016)
11. Cimatti, A., Franzén, A., Griggio, A., Sebastiani, R., Stenico, C.: Satisfiability modulo the theory of costs: foundations and applications. In: Esparza, J., Majumdar, R. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, pp. 99–113. Springer, Berlin Heidelberg, Berlin, Heidelberg (2010)
12. Cimatti, A., Griggio, A., Schaafsma, B.J., Sebastiani, R.: A modular approach to MaxSAT modulo theories. In: Järvisalo, M., Van Gelder, A. (eds.) SAT 2013. LNCS, vol. 7962, pp. 150–165. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39071-5_12
13. Craciunas, S.S., Oliver, R.S., Chmelík, M., Steiner, W.: Scheduling real-time communication in IEEE 802.1qbv time sensitive networks. In: Proceedings of the 24th International Conference on Real-Time Networks and Systems, p. 183-192. RTNS 2016, Association for Computing Machinery, New York, NY, USA (2016)
14. Demarchi, S., Menapace, M., Tacchella, A.: Automating elevator design with satisfiability modulo theories. In: 2019 IEEE 31st International Conference on Tools with Artificial Intelligence (ICTAI), pp. 26–33 (2019)
15. Demarchi, S., Tacchella, A., Menapace, M.: Automated design of complex systems with constraint programming techniques. In: Proceedings of the Cyber-Physical

Systems Ph.D Workshop 2019, CPS Summer School "Designing Cyber-Physical Systems - From concepts to implementation", Alghero, Italy, pp. 51–59 (2019)
16. Dutertre, B., de Moura, L.: Integrating simplex with DPLL(T). Technical Report, SRI International (2006)
17. Eraşcu, M., Micota, F., Zaharie, D.: Applying optimization modulo theory, mathematical programming and symmetry breaking for automatic deployment in the cloud of component-based applications extended abstract. In: 4th Women in Logic Workshop, p. 6 (2020)
18. Erata, F., Piskac, R., Mateu, V., Szefer, J.: Towards automated detection of single-trace side-channel vulnerabilities in constant-time cryptographic code (2023). arXiv preprint arXiv:2304.02102
19. Fazekas, K., Bacchus, F., Biere, A.: Implicit hitting set algorithms for maximum satisfiability modulo theories. In: Galmiche, D., Schulz, S., Sebastiani, R. (eds.) IJCAR 2018. LNCS (LNAI), vol. 10900, pp. 134–151. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-94205-6_10
20. Feng, J., Zhang, T., Yi, C.: Reliability-aware comprehensive routing and scheduling in time-sensitive networking. In: Wireless Algorithms, Systems, and Applications: 17th International Conference, WASA 2022, Dalian, China, November 24–26, 2022, Proceedings, Part II, pp. 243–254. Springer (2022)
21. Gavran, I., Darulova, E., Majumdar, R.: Interactive synthesis of temporal specifications from examples and natural language. In: Proceedings of the ACM on Programming Languages, vol. 4(OOPSLA), pp. 1–26 (2020)
22. Gretsch, R., Song, P., Madhavan, A., Lau, J., Sherwood, T.: Energy efficient convolutions with temporal arithmetic. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 2, pp. 354–368. ASPLOS 2024, Association for Computing Machinery, New York, NY, USA (2024)
23. Henry, J., Asavoae, M., Monniaux, D., Maïza, C.: How to compute worst-case execution time by optimization modulo theory and a clever encoding of program semantics. In: Proceedings of the 2014 SIGPLAN/SIGBED Conference on Languages, Compilers and Tools for Embedded Systems, Edinburgh, United Kingdom, pp. 43–52. Association for Computing Machinery, New York, NY, USA (2014)
24. Jackson, D., Estler, H.C., Rayside, D.: The Guided Improvement Algorithm for Exact, General-Purpose, Many-Objective Combinatorial Optimization. Technical Report 2009-033, MIT-CSAIL (2009)
25. Jiang, J., Chen, L., Wu, X., Wang, J.: Block-wise abstract interpretation by combining abstract domains with SMT. In: Bouajjani, A., Monniaux, D. (eds.) Verification, Model Checking, and Abstract Interpretation - 18th International Conference, VMCAI 2017, Paris, France, January 15-17, 2017, Proceedings. Lecture Notes in Computer Science, vol. 10145, pp. 310–329. Springer (2017). https://doi.org/10.1007/978-3-319-52234-0_17
26. Jin, X., Xia, C., Guan, N., Zeng, P.: Joint algorithm of message fragmentation and no-wait scheduling for time-sensitive networks. IEEE/CAA J. Automatica Sin. **8**(2), 478–490 (2021)
27. Jovanović, D., de Moura, L.: Solving non-linear arithmetic. In: Gramlich, B., Miller, D., Sattler, U. (eds.) IJCAR 2012. LNCS (LNAI), vol. 7364, pp. 339–354. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-31365-3_27
28. Karpenkov, G.E.: Finding inductive invariants using satisfiability modulo theories and convex optimization, Ph.D Thesis. Université Grenoble Alpes (2017). https://tel.archives-ouvertes.fr/tel-01681555/file/KARPENKOV_2017_diffusion.pdf

29. Knüsel, M.: Optimizing Declarative Power Sequencing. Master thesis, ETH Zurich, Zurich (2021-09)
30. Kovásznai, G., Biró, C., Erdélyi, B.: Puli - a problem-specific OMT solver. EasyChair Preprints (2018)
31. Krstić, S., Goel, A.: Architecting solvers for SAT modulo theories: Nelson-Oppen with DPLL. In: Konev, B., Wolter, F. (eds.) FroCoS 2007. LNCS (LNAI), vol. 4720, pp. 1–27. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-74621-8_1
32. Lai, L., Fiaschi, L., Cococcioni, M., Deb, K.: Pure and mixed lexicographic-paretian many-objective optimization: state of the art. Nat. Comput. Int. J. **22**(2), 227-242 (2022)
33. Larraz, D., Oliveras, A., Rodríguez-Carbonell, E., Rubio, A.: Minimal-model-guided approaches to solving polynomial constraints and extensions. In: Sinz, C., Egly, U. (eds.) Theory and Applications of Satisfiability Testing - SAT 2014, pp. 333–350. Springer International Publishing, Cham (2014)
34. Lee, D., et al.: SP&R: SMT-based simultaneous place-and-route for standard cell synthesis of advanced nodes. IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. **40**(10), 2142–2155 (2020)
35. Leofante, F., Giunchiglia, E., Ábrahám, E., Tacchella, A.: Optimal planning modulo theories. In: Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence, pp. 4128–4134 (2021)
36. Li, Y., Albarghouthi, A., Kincaid, Z., Gurfinkel, A., Chechik, M.: Symbolic optimization with SMT solvers. In: Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Diego, California, USA, pp. 607–618 (2014). https://doi.org/10.1145/2535838.2535857
37. Liu, T., Tyszberowicz, S., Beckert, B., Taghdiri, M.: Computing exact loop bounds for bounded program verification. In: Larsen, K.G., Sokolsky, O., Wang, J. (eds.) SETTA 2017. LNCS, vol. 10606, pp. 147–163. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-69483-2_9
38. Marchetto, G., Sisto, R., Valenza, F., Yusupov, J., Ksentini, A.: A formal approach to verify connectivity and optimize VNF placement in industrial networks. IEEE Trans. Industr. Inf. **17**(2), 1515–1525 (2021)
39. de Moura, L.M., Bjørner, N.S.: Model-based theory combination. In: Krstic, S., Oliveras, A. (eds.) Proceedings of the 5th International Workshop on Satisfiability Modulo Theories, SMT@CAV 2007, Berlin, Germany, July 1-2, 2007. Electronic Notes in Theoretical Computer Science, vol. 198, pp. 37–49. Elsevier (2007)
40. Nadel, A., Ryvchin, V.: Bit-vector optimization. In: Chechik, M., Raskin, J.-F. (eds.) TACAS 2016. LNCS, vol. 9636, pp. 851–867. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-49674-9_53
41. Nguyen, C.M., Sebastiani, R., Giorgini, P., Mylopoulos, J.: Requirements evolution and evolution requirements with constrained goal models. In: Comyn-Wattiau, I., Tanaka, K., Song, I.-Y., Yamamoto, S., Saeki, M. (eds.) ER 2016. LNCS, vol. 9974, pp. 544–552. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-46397-1_42
42. Nguyen, C.M., Sebastiani, R., Giorgini, P., Mylopoulos, J.: Modeling and reasoning on requirements evolution with constrained goal models. In: Cimatti, A., Sirjani, M. (eds.) Software Engineering and Formal Methods - 15th International Conference, SEFM 2017, Trento, Italy, September 4-8, 2017, Proceedings. Lecture Notes in Computer Science, vol. 10469, pp. 70–86. Springer (2017)

43. Nguyen, C.M., Sebastiani, R., Giorgini, P., Mylopoulos, J.: Multi-objective reasoning with constrained goal models. Requirements Eng. **23**(2), 189–225 (2016). https://doi.org/10.1007/s00766-016-0263-5
44. Nieuwenhuis, R., Oliveras, A.: On SAT modulo theories and optimization problems. In: Biere, A., Gomes, C.P. (eds.) Theory and Applications of Satisfiability Testing - SAT 2006, pp. 156–169. Springer, Berlin Heidelberg, Berlin, Heidelberg (2006)
45. Nieuwenhuis, R., Oliveras, A., Tinelli, C.: Solving SAT and SAT Modulo Theories: from an abstract Davis-Putnam-Logemann-Loveland Procedure to DPLL(T). J. ACM **53**(6), 937–977 (2006)
46. Paoletti, N., et al.: Synthesizing stealthy reprogramming attacks on cardiac devices. In: Proceedings of the 10th ACM/IEEE International Conference on Cyber-Physical Systems, ICCPS 2019, pp. 13–22. Association for Computing Machinery, New York, NY, USA (2019)
47. Park, D., Lee, D., Kang, I., Gao, S., Lin, B., Cheng, C.K.: SP&R: simultaneous placement and routing framework for standard cell synthesis in sub-7nm. In: 2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC), pp. 345–350 (2020)
48. Patti, G., Bello, L.L., Leonardi, L.: Deadline-aware online scheduling of tsn flows for automotive applications. IEEE Trans. Ind. Inform. **19**(4), 5774–5784 (2022)
49. Ratschan, S.: Simulation based computation of certificates for safety of dynamical systems (2017). arXiv preprint arXiv:1707.00879
50. Roc, O.V.: Optimization Modulo Theories. Master's thesis, Polytechnic University of Catalonia. https://upcommons.upc.edu/handle/2099.1/14204 (2011)
51. Rybalchenko, A., Vuppalapati, C.: Supercharging plant configurations using z3. In: Integration of Constraint Programming, Artificial Intelligence, and Operations Research: 18th International Conference, CPAIOR 2021, Vienna, Austria, July 5–8, 2021, Proceedings. vol. 12735, p. 1. Springer Nature (2021)
52. Schmitt, T., Hoffmann, M., Rodemann, T., Adamy, J.: Incorporating human preferences in decision making for dynamic multi-objective optimization in model predictive control. Inventions **7**(3), 46 (2022)
53. Sebastiani, R., Trentin, P.: Pushing the envelope of optimization modulo theories with linear-arithmetic cost functions. In: Baier, C., Tinelli, C. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, pp. 335–349. Springer, Berlin Heidelberg, Berlin, Heidelberg (2015)
54. Sebastiani, R., Tomasi, S.: Optimization in SMT with $\mathcal{LA}(\mathbb{Q})$ cost functions. In: Gramlich, B., Miller, D., Sattler, U. (eds.) Automated Reasoning, pp. 484–498. Springer, Berlin Heidelberg, Berlin, Heidelberg (2012)
55. Sebastiani, R., Tomasi, S.: Optimization modulo theories with linear rational costs. ACM Trans. Comput. Logic **16**(2), 1–43 (2015)
56. Sebastiani, R., Trentin, P.: OptiMathSAT: a tool for optimization modulo theories. J. Autom. Reasoning **64**(3), 423–460 (2018). https://doi.org/10.1007/s10817-018-09508-6
57. Sebastiani, R., Trentin, P.: On optimization modulo theories, maxSMT and sorting networks. In: Legay, A., Margaria, T. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, pp. 231–248. Springer, Berlin Heidelberg, Berlin, Heidelberg (2017)
58. Shen, D., Zhang, T., Wang, J., Deng, Q., Han, S., Hu, X.S.: QoS guaranteed resource allocation for coexisting eMBB and URLLC traffic in 5G industrial networks. In: 2022 IEEE 28th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA), pp. 81–90. IEEE (2022)

59. Sivaraman, A., Farnadi, G., Millstein, T., Van den Broeck, G.: Counterexample-guided learning of monotonic neural networks. Adv. Neural. Inf. Process. Syst. **33**, 11936–11948 (2020)
60. Subramanian, S., Berzish, M., Tripp, O., Ganesh, V.: A solver for a theory of strings and bit-vectors. In: 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C), pp. 124–126 (2017)
61. Tarrach, T., Ebrahimi, M., König, S., Schmittner, C., Bloem, R., Nickovic, D.: Thorsten Tarrach and Masoud Ebrahimi and Sandra König and Christoph Schmittner and Roderick Bloem and Dejan Nickovic, WorkingPaper (2022)
62. Teso, S., Sebastiani, R., Passerini, A.: Structured learning modulo theories. Artif. Intell. **244**, 166–187 (2017)
63. Tierno, A., Turri, G., Cimatti, A., Passerone, R.: Symbolic encoding of reliability for the design of redundant architectures. In: 2022 IEEE 5th International Conference on Industrial Cyber-Physical Systems (ICPS), pp. 01–06 (2022)
64. Trentin, P.: Optimization Modulo Theories with OptiMathSAT. Ph.D. thesis, University of Trento (2019)
65. Trentin, P., Sebastiani, R.: Optimization modulo the theories of signed bit-vectors and floating-point numbers. J. Autom. Reason. **65**(7), 1071-1096 (2021)
66. Trentin, P., Sebastiani, R.: Optimization modulo the theories of signed bit-vectors and floating-point numbers. J. Autom. Reason. **65**(7), 1071–1096 (2021)
67. Tsiskaridze, N., Barrett, C., Tinelli, C.: Generalized optimization modulo theories (2024). arXiv preprint arXiv:2404.16122
68. Tsiskaridze, N., et al.: Automating system configuration. In: Formal Methods in Computer Aided Design, FMCAD 2021, New Haven, CT, USA, October 19-22, 2021, pp. 102–111. IEEE (2021)
69. Yao, P., Shi, Q., Huang, H., Zhang, C.: Program analysis via efficient symbolic abstraction. In: Proceedings of the ACM on Programming Languages, vol. 5(OOPSLA), pp. 1–32 (2021)

**Open Access** This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

